# MineOps

## Developer Guide

How the Code is Organised and How to Work With It

| | |
|---|---|
| **Group** | Syntax Crew |
| **Module** | I3691CP |
| **Semester** | 1, 2026 |
| **Version** | 1.0 |
| **Prepared by** | Project Manager: Katare Nderura |

JEDS Campus — School of Engineering

## 1. Who This Guide Is For

This guide is written for any Syntax Crew developer — or anyone new to the MineOps codebase — who needs to understand how the code is structured, how the different parts connect to Firebase, and how to add a new feature without breaking what is already working. Read the **Setup Guide** first to get the app running. Then read this guide before you write any code.

## 2. The Big Picture — How the App is Structured

MineOps is a React Native app built with Expo. Every screen the user sees is a JavaScript file in the `src/screens/` folder. Those screens talk to Firebase through service files in `src/services/`. The navigation (which screens you see and in what order) is defined in `src/navigation/`. No screen is allowed to talk to Firebase directly — all Firebase calls go through the service files.

> **RULE:** Never write a Firestore query or a Firebase Auth call directly inside a screen file. Always write it in the correct service file and call it from the screen. This keeps the code clean and makes it easy to fix bugs.

### 2.1 Folder Map

primary

| | |
|---|---|
| lightgray `src/screens/` | One `.js` file per screen. A screen is a full page the user sees (e.g. `LoginScreen.js`, `DashboardScreen.js`). Screens contain JSX (the visual layout) and call service functions to get or save data. |
| `src/components/` | Reusable building blocks. If the same UI element appears on more than one screen (e.g. a `TaskCard`, a `HazardBadge`, a `PriorityTag`), it lives here as its own component file. |
| lightgray `src/services/` | One `.js` file per Firebase feature. `authService.js` handles login/logout/register. `taskService.js` handles all task reads and writes. `hazardService.js` handles hazard flags. These files are the **ONLY** place Firebase code should appear. |
| `src/config/firebaseConfig.js` | Initialises the Firebase app and exports `auth`, `db` (Firestore), and `storage`. Every service file imports from here. This file is in `.gitignore` — **never commit it**. |
| lightgray `src/navigation/` | Defines how screens connect. `AppNavigator.js` sets up the bottom tab navigator and the authentication stack. If you add a new screen, you register it here. |
| `src/hooks/` | Custom React hooks. For example, `useAuth.js` exposes the current logged-in user to any screen that needs it. |
| lightgray `assets/` | Static files: app icon (`icon.png`), splash screen (`splash.png`), and any images used in the UI. |

primary

| | |
|---|---|
| `app.json` | Expo configuration. Do not edit this unless you are changing the app name, version, or build settings — ask the GitHub Managers first. |
| lightgray `docs/` | All documentation. The SRS, this guide, the setup guide, the feature register, and the progress tracker all live here. |
| `designs/` | PNG exports of every Figma screen. Developers use these as their visual reference when building screens. |

## 3. Firebase — How MineOps Uses It

### 3.1 Firebase Authentication

Firebase Auth manages who is logged in. The app uses email/password login. After a user logs in, Firebase gives back a user object that contains the user's UID (a unique ID string). That UID is how MineOps links a logged-in person to their record in the `users` Firestore collection.

All auth operations live in `src/services/authService.js`. Here is the pattern:

```
// src/services/authService.js
import { auth } from '../config/firebaseConfig';
import { createUserWithEmailAndPassword, signInWithEmailAndPassword, signOut
    } from 'firebase/auth';

// Register a new user
export const registerUser = (email, password) => {
  return createUserWithEmailAndPassword(auth, email, password);
};

// Log in
export const loginUser = (email, password) => {
  return signInWithEmailAndPassword(auth, email, password);
};

// Log out
export const logoutUser = () => signOut(auth);
```

### 3.2 Firestore — Reading and Writing Data

Firestore stores all the app's data: tasks, hazard flags, handover reports, notifications, and audit logs. Data is organised as *collections* (like database tables) containing *documents* (like individual records).

**Example — reading all tasks assigned to the current user:**

```
// src/services/taskService.js
import { db } from '../config/firebaseConfig';
import { collection, query, where, orderBy, getDocs } from
    'firebase/firestore';

export const getMyTasks = async (userId) => {
  const q = query(
    collection(db, 'tasks'),
    where('assignedTo', '==', userId),
    orderBy('priority'),
    orderBy('dueDate')
```

```
  );
  const snapshot = await getDocs(q);
  return snapshot.docs.map(doc => ({ id: doc.id, ...doc.data() }));
};
```

**Example — creating a new task:**

```
// src/services/taskService.js (continued)
import { addDoc, serverTimestamp } from 'firebase/firestore';

export const createTask = async (taskData) => {
  return await addDoc(collection(db, 'tasks'), {
    ...taskData,
    status: 'Not Started',
    createdAt: serverTimestamp(),
    updatedAt: serverTimestamp(),
  });
};
```

### 3.3 Firebase Storage — Photo Uploads

When a user attaches a photo to a hazard report or task update, the image is uploaded to Firebase Storage. The download URL is then saved in the Firestore document so it can be displayed later.

```
// src/services/storageService.js
import { storage } from '../config/firebaseConfig';
import { ref, uploadBytes, getDownloadURL } from 'firebase/storage';

export const uploadPhoto = async (uri, path) => {
  const response = await fetch(uri);
  const blob = await response.blob();
  const storageRef = ref(storage, path);
  await uploadBytes(storageRef, blob);
  return await getDownloadURL(storageRef);
};
```

### 3.4 Firestore Security Rules

Firestore security rules control who can read and write what data. These rules run on Firebase's servers — they cannot be bypassed by app code. The basic rule for MineOps is that every read and write requires the user to be logged in:

```
// Firebase Console -> Firestore -> Rules
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /{document=**} {
      allow read, write: if request.auth != null;
    }
  }
}
```

> **IMPORTANT:** These rules must be set in the Firebase Console before the app demo. A Firebase Engineer is responsible for configuring the rules. If the rules are not set, anyone with the API key can read your data — **this will cost marks**.

## 4. How to Add a New Screen

Follow these steps every time you add a new screen to MineOps:

1. **Create a new file** in `src/screens/`. Name it exactly after what the screen does.
   *Example:* for the compliance report export screen, create `src/screens/ComplianceExportScreen.js`

2. **Write the screen component.** Every screen is a React function component that returns JSX. Start with a basic structure and build up from there. Use `StyleSheet.create()` for all styles — never use inline style objects.

3. **Write Firebase functions in the service file first** (e.g. `taskService.js` for task data). Import and call those functions from the screen. Never write Firebase code directly in the screen.

4. **Register the new screen** in `src/navigation/AppNavigator.js`. Add it to the correct navigator — the main tab navigator if it is a primary tab, or a stack navigator if it is a sub-screen reached from another screen.

5. **Check the Figma prototype** in `designs/` to see what the screen should look like. Match the layout, colours, font sizes, and component structure as closely as possible.

6. **Test on a physical device** before committing. Make sure the screen works, the back navigation works, and no errors appear in the Expo terminal.

7. **Commit with a descriptive message:**

```
feat: Add ComplianceExportScreen with Firestore date-range filter
```

## 5. How User Roles Work in the Code

When a user logs in, the app reads their role from the `users` Firestore collection using their UID. The role is stored in a context (or a global state) that every screen can access. Screens use the role to show or hide features.

**Example — checking role before allowing task creation:**

```
// src/hooks/useAuth.js
import { auth, db } from '../config/firebaseConfig';
import { onAuthStateChanged } from 'firebase/auth';
import { doc, getDoc } from 'firebase/firestore';
import { useState, useEffect } from 'react';

export const useAuth = () => {
  const [user, setUser] = useState(null);
  const [role, setRole] = useState(null);

  useEffect(() => {
    const unsub = onAuthStateChanged(auth, async (firebaseUser) => {
      if (firebaseUser) {
        setUser(firebaseUser);
        const userDoc = await getDoc(doc(db, 'users', firebaseUser.uid));
        setRole(userDoc.data()?.role || 'Operator');
      } else {
        setUser(null); setRole(null);
      }
    });
    return unsub;
```

```
  }, []);

  return { user, role };
};
```

**Then in any screen:**

```
const { user, role } = useAuth();

// Only show Create Task button to Supervisors and Admins
{(role === 'Supervisor' || role === 'Admin') && (
  <TouchableOpacity onPress={handleCreateTask}>
    <Text>Create Task</Text>
  </TouchableOpacity>
)}
```

## 6. Code Standards — Rules All Developers Must Follow

primary

| | |
|---|---|
| lightgray All Firebase calls go in `/services/` files — never in screens | Keeps the codebase clean. If Firebase changes, you fix it in one place instead of hunting through every screen. |
| Use `StyleSheet.create()` for all styles | Styles defined this way are validated at startup and perform better on Android. |
| lightgray Name files exactly after what they do | `DashboardScreen.js`, `taskService.js`. Anyone reading the folder structure should know exactly what each file does. |
| Use meaningful variable names | `const taskList` is fine. `const x` and `const data` are not — nobody knows what they hold. |
| lightgray Never commit `firebaseConfig.js` to GitHub | It contains API keys. It is in `.gitignore` but always double-check with `git status` before pushing. |
| Write a descriptive commit message for every commit | `feat: Add real-time listener for tasks collection` is good. `update` is not. |
| lightgray Test on a real device before pushing | The Expo emulator does not perfectly simulate all device behaviours. Always test Firebase sync and notifications on a real phone. |
| Pull from GitHub before starting work each day | `git pull` — avoids merge conflicts with your teammates' changes. |

## 7. Contacts — Who to Ask for What

primary

| | | |
|---|---|---|
| lightgray Firebase Auth or Firestore not connecting | Saara Ndweda, Ndapandula Glukua, or Gehas Imene | 812754954 / 813988029 / 813925804 |
| GitHub push/pull conflicts or repository access | Feliciano Nakandjibi or Frans Andreas | 817748965 / 816344346 |
| lightgray Figma design questions — what should a screen look like | Eliazer Katondoka, Niinkoti Tomas, or Linea Shevaanyena | 814424800 / 813193491 / 815805405 |
| Documentation — what should be written in the docs | Shatika Titus, Masaku Fernandu, or Shimakeleni Johannes | 816540054 / 813347467 / 818779180 |
| lightgray Project scope, deadline, or marking questions | Katare Nderura or Amwaama Ebba (Project Managers) | 818161664 / 814389055 |
| Lecturer enquiry — scope change, extension, integrity issue | Mr. Mathew Abisai — JEDS Campus, School of Engineering | — |